RETURN[ER] MACHINE · FORMAT .RMK · RMH_KONTEKS.PY ORDE 3

Di Dalam File .rmk

GPU `List.txt` berukuran **136.412 byte** menjadi **12.531 byte**. Di dalamnya tidak ada histogram dan tidak ada tabel. Isinya hanya 12 byte kepala dan satu deretan bit. Halaman ini menunjukkan dari mana deretan bit itu berasal.

Isi file, byte demi byte

Inilah 12 byte pertama gpu `.rmk` dan awal kodenya, persis seperti yang tersimpan di disk.



magic	n	orde	crc32
"RMK1"	136.412 byte	3	checksum



kode
12.519 byte, satu deretan bit untuk seluruh file

0 byte untuk histogram dan konteks. Semuanya dibangun ulang oleh decoder.

```
asli
136.412 byte


.rmb
89.892 byte


.rmk
12.531 byte
```

Digambar sesuai skala. Pada `.rmb`, setiap blok 4.096 byte menyimpan histogramnya sendiri (ungu) dan rank (biru), 34 kali. Pada `.rmk` hanya ada satu deretan kode.

Satu byte, satu tebakan

Setiap byte disimpan dengan cara yang sama. Contoh di bawah diambil dari file ini: setelah huruf `tio`, model sudah 2.877 kali melihat huruf n.

1 · lihat ke belakang

`"tio"`

Tiga byte sebelumnya menjadi konteks.

2 · buka histogram konteks

`n : 2.877`

Histogram ini tidak ada di file. Model membangunnya sendiri dari byte yang sudah lewat.

3 · tebak

`n ≈ 99,98%`

Makin yakin tebakannya, makin sedikit bit yang dibutuhkan.

4 · tulis bit

`≈ 0,0003 bit`

Byte 8 bit cukup disimpan dengan sepersekian bit di deretan kode.

Encoder (saat menyimpan)

```
byte asli → model menebak → bit
lalu model belajar dari byte itu
```

Decoder (saat membuka)

```
bit → model yang sama menebak → byte asli
lalu model belajar dari byte itu
```

Kalau byte belum pernah muncul di konteks itu, encoder menulis tanda **ESCAPE**, lalu mencoba konteks yang lebih pendek (2 byte, 1 byte, 0 byte). Kalau tetap tidak ada, byte itu dianggap **baru** dan butuh sekitar 8 bit.

Coba sendiri

Ketik apa saja. Setiap kotak adalah satu byte, dan warnanya menunjukkan berapa bit yang dibutuhkan untuk menyimpannya. Pilih sebuah kotak untuk melihat langkah tebakannya. Model di sini sama persis dengan `rmh_konteks.py`.

Teks

bismillahirrahmanirrahim. kata yang sering muncul akan semakin murah. kata yang sering muncul akan semakin murah. kata yang sering muncul akan semakin murah!

Orde maksimum

3

asli **157 byte** = 1.256 bit kode **469,9 bit** ≈ 58,7 byte

rata-rata **2,99 bit/byte** **37,4%** dari asli

bit per byte: ☐ < 0,1 ☐ 0,1–0,5 ☐ 0,5–1 ☐ 1–2 ☐ 2–4 ☐ 4–8 ☐ ≥ 8

b i s m i l l a h i r r a h m a n i r r a h
i m . ␣ k a t a ␣ y a n g ␣ s e r i n g ␣ m
u n c u l ␣ a k a n ␣ s e m a k i n ␣ m u r
a h . ␣ k a t a ␣ y a n g ␣ s e r i n g ␣ m
u n c u l ␣ a k a n ␣ s e m a k i n ␣ m u r
a h . ␣ k a t a ␣ y a n g ␣ s e r i n g ␣ m
u n c u l ␣ a k a n ␣ s e m a k i n ␣ m u r
a h !

Byte ke-156: '!' → 13.43 bit

orde 3 · konteks "rah" ESCAPE · 1.42 bit, turun ke orde 2

m i . ESC

orde 2 · konteks "ah"

konteks ini belum pernah terlihat, dilewati tanpa biaya

orde 1 · konteks "h"

konteks ini belum pernah terlihat, dilewati tanpa biaya

orde 0 · konteks "" ESCAPE · 4.12 bit, turun ke orde -1

a r n ␣ k u ☐

orde -1 · semua byte berpeluang sama

RAPU · 1 dari 238 kemungkinan · 7,89 bit

byte ini belum pernah muncul di teks sebelumnya

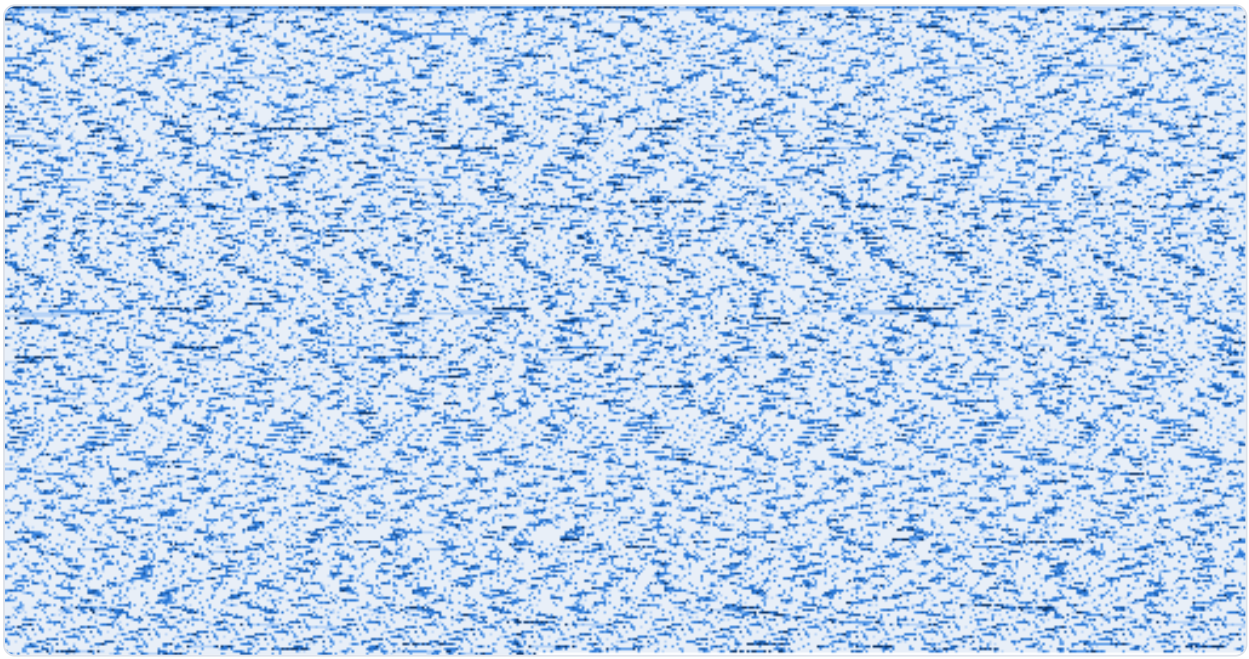
Lebar setiap bagian = peluang menurut histogram konteks saat itu. Bagian oranye adalah yang terjadi. Bit = log2(total / lebar bagian).

Peta piksel GPU List.txt

Setiap piksel adalah satu byte dari 136.412 byte file asli, dibaca dari kiri ke kanan dan dari atas ke bawah, 512 byte per baris. Warnanya menunjukkan berapa bit yang dihabiskan byte itu di `gpu.rmk`. Arahkan atau ketuk peta untuk melihat byte dan teks di sekitarnya.

Biaya bit | Orde penebak

bit per byte: < 0,1 | 0,1–0,5 | 0,5–1 | 1–2 | 2–4 | 4–8 | ≥ 8



byte **100.000** '6' konteks "="1" ditebak di **orde 3** **6,44 bit**

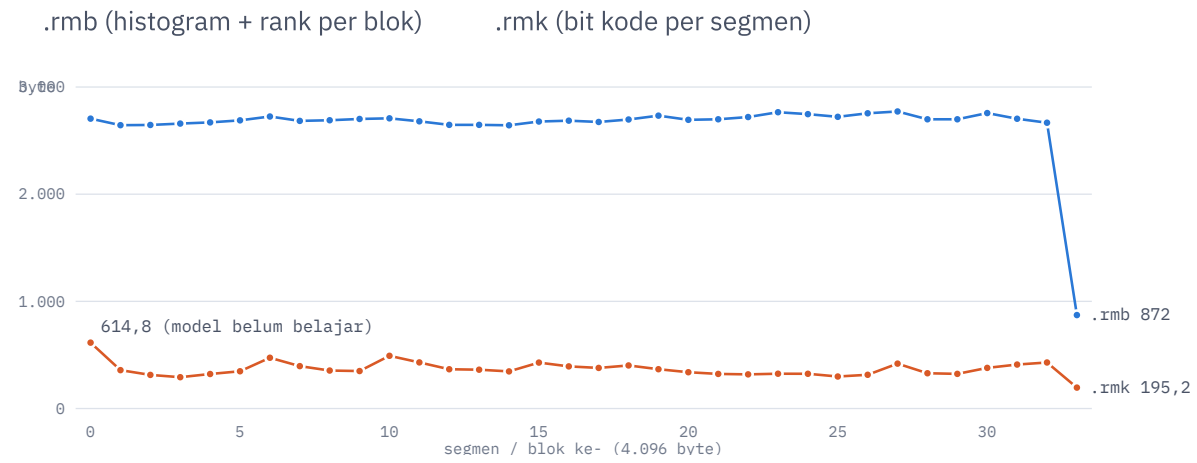
```
e ␣ G T X ␣ 2 8 5 M < / o p t i o n ␍ ↵ < o
p t i o n ␣ v a l u e = " 1 6 4 3 " ␣ c l a
s s = " b l _ b e n c h m a r k _ o p t i o
n _ 1 6 4 3
```

Latar yang terang adalah HTML berulang seperti `<option value="..." class="bl_benchmark_option_..."`, yang hampir tidak butuh bit. Titik dan garis biru tua adalah angka ID dan nama GPU yang sulit ditebak. Polanya miring karena setiap

baris `<option>` panjangnya sedikit berbeda, sehingga posisi angka bergeser sedikit demi sedikit dari baris ke baris. Pita tipis yang lebih gelap di paling atas adalah awal file, saat model belum belajar apa-apa.

Model belajar, blok tidak

Biaya setiap 4.096 byte di kedua format. Blok `.rmb` selalu mulai dari nol, jadi setiap blok membayar sekitar 2.650 byte. `.rmk` membayar mahal hanya di awal, lalu terus memakai apa yang sudah dipelajari.



▶ Lihat sebagai tabel

Ke mana 12.519 byte kode pergi

Setiap bit di deretan kode bisa dilacak asalnya. Lebih dari separuh dipakai untuk byte yang berhasil ditebak dari 3 byte sebelumnya, meskipun rata-rata setiap byte itu hanya butuh setengah bit.

byte di orde 3 **8.046,3 byte · 64,3%** 130.468 kali, rata-rata 0,49 bit	byte di orde 2 **1.099,4 byte · 8,8%** 3.201 kali, rata-rata 2,75 bit	byte di orde 1 **965,7 byte · 7,7%** 1.986 kali, rata-rata 3,89 bit
byte di orde 0 **536,7 byte · 4,3%** 682 kali, rata-rata 6,30 bit	byte baru **72,8 byte · 0,6%** 75 kali, rata-rata 7,77 bit	☐ ESCAPE **1.797,2 byte · 14,4%** 5.364 kali, rata-rata 2,68 bit

Hasil akhir

GPU List.txt

asli	136.412
.rmb	89.892